AI-Based Detection of Malicious Sideloaded Android Applications

System Architecture, Design Flow, Dataflow & Technical Specifications

Executive Summary

Sideloaded Android applications—installed outside official app stores like Google Play—bypass pre-deployment vetting, exposing users to sophisticated adware, spyware, banking Trojans, and root exploits. Modern Android malware routinely employs obfuscation and dynamic payload loading that readily defeat signature-based static antiviruses.

This system provides a holistic, multi-tier defense-in-depth architecture combining: (1) safe static APK decompilation and 215 Drebin feature extraction, (2) Random Forest and XGBoost soft voting ensemble achieving **98.14% test accuracy**, (3) local Explainable AI via SHAP TreeExplainer, (4) automated runtime threat intelligence via AbuseIPDB API v2, (5) dynamic Android sandbox emulation capturing 139 Linux kernel system calls, and (6) a modern reactive Streamlit web dashboard.

KEY HIGHLIGHT: Dual static-dynamic synergy augmented by runtime threat intelligence and local Shapley feature attribution delivers both high detection accuracy (98.14% static / 97.31% dynamic) and verifiable human-in-the-loop auditability.

1. System Architecture & High-Level Design Flow

The architecture is organized into five decoupled tiers ensuring fault tolerance, strict isolation, and high modularity:

• **1. Presentation Tier (Streamlit Web Dashboard):** Provides reactive tabs for single-click APK triage, model evaluation metrics, live dynamic sandbox execution, and threat intelligence visualization.

• **2. Static Ingestion & Bytecode Engine:** Uses Androguard 4.x to inspect AndroidManifest.xml and classes.dex bytecode without executing untrusted code.

• **3. Static ML & Explainability Engine:** Evaluates the 215-dimensional Drebin binary vector across Random Forest and XGBoost, computing soft ensemble probabilities and SHAP TreeExplainer attributions.

• **4. Threat Intelligence Engine:** Harvests embedded public IPv4 addresses and domain hostnames, filters RFC 1918 private scopes, performs live DNS lookups, and queries AbuseIPDB v2.

• **5. Dynamic Emulation Sandbox:** Connects via ADB bridge to a rooted Android Virtual Device (AVD), injects the APK, fuzzes the UI with Monkey, and records 139 Linux kernel syscalls via strace.

2. Machine Learning & Deep Learning Algorithms

2.1 Static Analysis (Drebin-215)

The static pipeline trains on 15,036 application vectors (5,560 malware, 9,476 benign) deduplicated to 7,254 unique samples across 215 binary indicators (permissions, intents, API calls, commands). An ensemble soft-voting scheme fuses Random Forest (100 trees, Gini impurity) and XGBoost (learning rate 0.1, depth 6) via arithmetic probability averaging: $P_{ensemble} = 0.5 \cdot (P_{RF} + P_{XGB})$.

2.2 Dynamic Analysis (CICMalDroid2020)

Dynamic analysis evaluates 11,520 applications traced during runtime execution across 139 Linux kernel system calls. Models include tabular XGBoost, Multi-Branch Temporal Convolutional Network (TCN with dilated causal convolutions), Bidirectional LSTM (BiLSTM), and Multi-Head Self-Attention Transformer. Crucially, evaluation utilizes **Cluster-Held-Out cross-validation**, holding out entire malware families to prevent cross-family data leakage.

3. Experimental Results & Performance Benchmarks

Table 1: Static Analysis Evaluation (Drebin-215 Held-Out Test Split)

Model	Accuracy	Precision	Recall	F1-Score	ROC-AUC
Random Forest	97.11%	96.02%	91.55%	0.9373	0.9926
XGBoost	98.00%	95.91%	95.63%	0.9577	0.9961
Fused Ensemble (RF + XGB)	98.14%	96.75%	95.34%	0.9604	0.9952

Table 2: Dynamic System-Call Evaluation (CICMalDroid2020: Random vs. Cluster-Held-Out)

Model	Random Acc	Random ROC-AUC	Cluster-Held-Out Acc	Cluster-Held-Out ROC
XGBoost	97.31%	0.9915	91.99% ± 3.36%	0.9600 ± 0.0223
Transformer	90.93%	0.9691	86.15% ± 6.62%	0.9419 ± 0.0125
TCN (Multi-Branch)	94.53%	0.9554	84.99% ± 3.14%	0.9013 ± 0.0409
BiLSTM	90.80%	0.9523	79.69% ± 9.25%	0.8806 ± 0.0552

4. Benchmark Figures & Empirical Evidence

Figure 1: Static Feature Importance (Drebin-215)

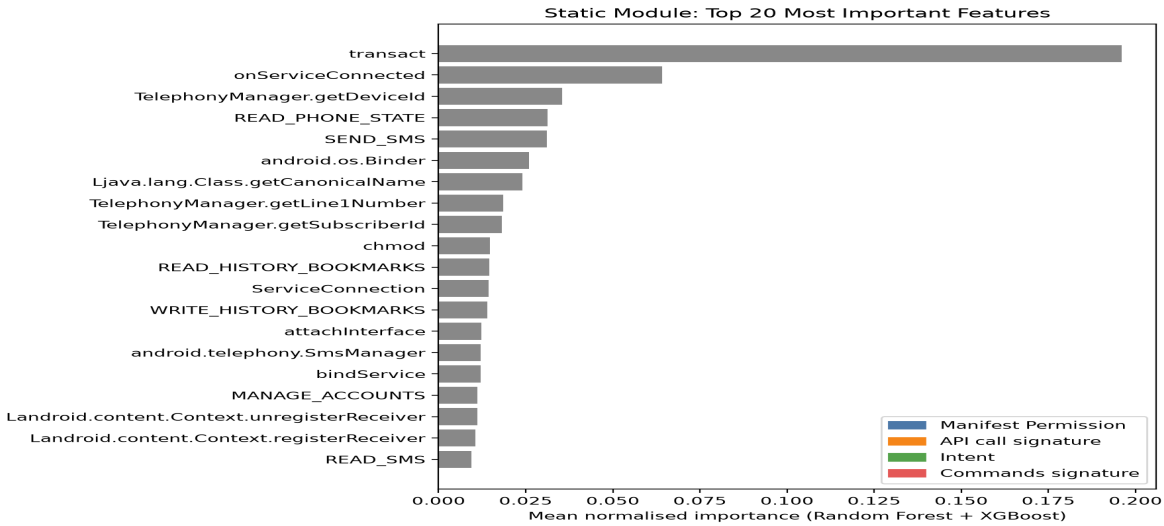


Figure 2: Dynamic ROC and Precision-Recall Curves (CICMalDroid2020)

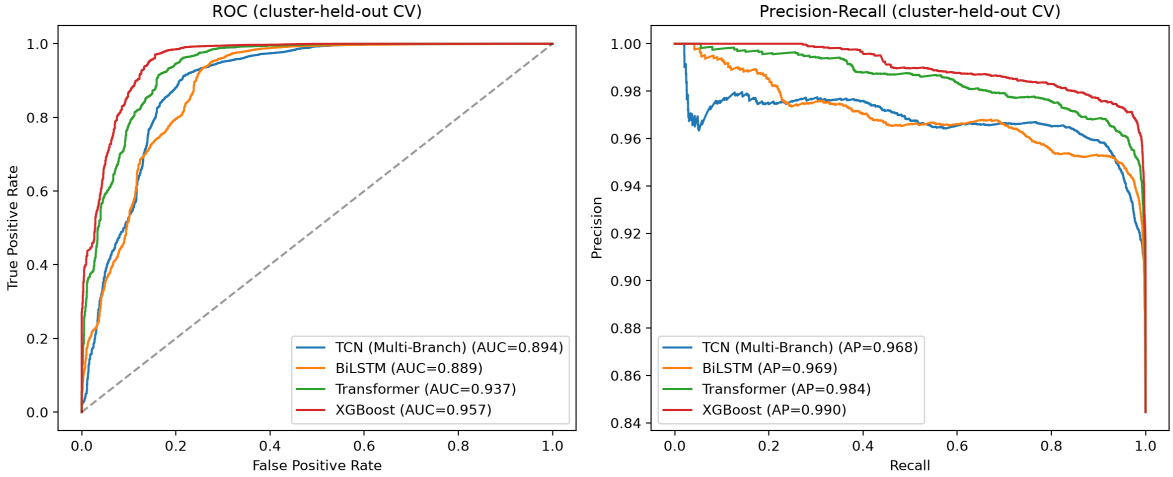
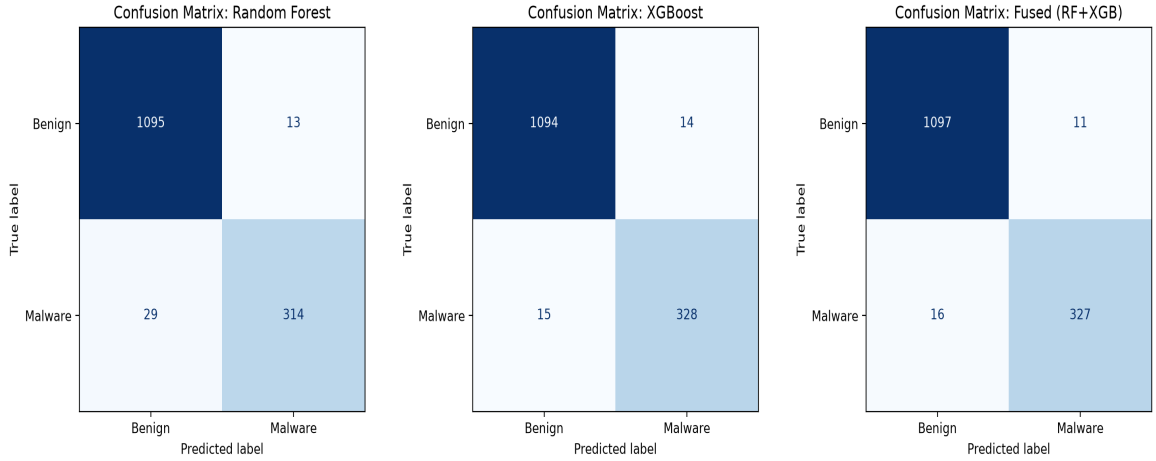


Figure 3: Confusion Matrices for Static Drebin Classifiers



5. Technical Specifications & Stack Details

Layer	Component	Version	Role
Host Platform	Windows 11 / WSL 2 Ubuntu	24.04 LTS	Virtualization, Python runtime & dev host
Language	Python (x86_64)	3.12+	Core computational pipeline & API clients
Frontend UI	Streamlit	1.40+	Reactive web dashboard & user console
Decompiler	Androguard	4.1.4+	APK manifest & DEX bytecode parsing
Tabular ML	Scikit-Learn & XGBoost	1.6+ / 2.1+	Random Forest & Gradient Boosted trees
Deep Learning	PyTorch	2.5+	TCN, BiLSTM, and Transformer inference
Explainability	SHAP	0.46+	Shapley TreeExplainer feature attributions
Device Bridge	Android ADB	35.0+	WSL2 to Windows Android emulator bridge
Dynamic Sandbox	Android Studio Emulator	API 34 (Google APIs)	Rooted virtual device with strace & monkey
Threat Intel	AbuseIPDB REST API v2	HTTPS JSON	Live C2 IP reputation & risk scoring

6. Input & Output Specifications

Inputs:
• Android Package (.apk) file: Valid ZIP archive containing AndroidManifest.xml and classes.dex bytecode.
• Analysis Duration: Integer slider (10 to 120 seconds, default 30s).
• Threat Intel API Key: Optional AbuseIPDB v2 key provided via .env, environment variable, or UI.

Outputs:
• Static Verdict Banner: High-visibility classification (BENIGN vs. MALWARE) with calibrated probability.
• Soft Voting Distribution: Disaggregated probabilities from Random Forest and XGBoost.
• SHAP Local Feature Attribution: Ranked horizontal bar chart identifying exact positive and negative drivers.
• Threat Intelligence Report: Harvested public IPs, domains, Abuse Confidence Scores (0-100%), and report counts.
• Dynamic Syscall Summary: Table of 139 system calls with top dynamic indicator rankings.
• Audit Report (JSON): Downloadable apk_analysis_report.json for enterprise SIEM ingestion.

7. Limitations & Academic Integrity Disclosures

1. **Dataset Temporality:** Drebin-215 reflects historical Android permissions; modern Android runtime permissions may differ.
2. **Anti-Analysis Evasion:** Advanced malware may detect emulator parameters (qemu, build properties) or sleep to evade dynamic trace.
3. **Architecture Syscall Aliases:** CICMalDroid2020 used 32-bit ARM names. Live x86_64 emulator traces use internal alias mapping (mmap->mmap2, lseek->_llseek).